```
import json
import math
import threading
import time
from collections import deque

import requests
import websocket
from flask import Flask, jsonify, render_template_string

app = Flask(__name__)

PRODUCT_ID = "BTC-USD"
REST_URL = "https://api.exchange.coinbase.com"
WS_URL = "wss://advanced-trade-ws.coinbase.com"

MAX_CANDLES = 300
STALE_SECONDS = 20

lock = threading.RLock()
candles = deque(maxlen=MAX_CANDLES)
ticks = deque(maxlen=1200)
trade_flow = deque(maxlen=5000)

current_price = None
last_update = 0.0
last_error = ""
last_source = "NINGUNA"
ws_connected = False
services_started = False

session = requests.Session()
session.headers.update({
    "User-Agent": "BitcoinRadarLive/5.0",
    "Accept": "application/json"
})

def ema(values, period):
    if not values:
        return None
    alpha = 2.0 / (period + 1)
    result = float(values[0])
    for value in values[1:]:
```

```
        result = alpha * float(value) + (1-alpha) * result
    return result

def rsi(values, period=14):
    if len(values) < period + 1:
        return None
    changes = [float(values[i]) - float(values[i - 1]) for i in range(1, len(values))]
    gains = [max(change, 0.0) for change in changes]
    losses = [max(-change, 0.0) for change in changes]
    avg_gain = sum(gains[:period]) / period
    avg_loss = sum(losses[:period]) / period
    for i in range(period, len(changes)):
        avg_gain = ((avg_gain * (period - 1)) + gains[i]) / period
        avg_loss = ((avg_loss * (period - 1)) + losses[i]) / period
    if avg_loss == 0:
        return 100.0 if avg_gain > 0 else 50.0
    rs = avg_gain / avg_loss
    return 100.0 - (100.0 / (1.0 + rs))

def vwap_from_candles(history):
    rows = [c for c in history if float(c.get("volume", 0) or 0) > 0]
    total_volume = sum(float(c["volume"]) for c in rows)
    if total_volume <= 0:
        return None
    weighted_price = sum(
        (float(c["high"]) + float(c["low"]) + float(c["close"])) / 3.0
        * float(c["volume"])
        for c in rows
    )
    return weighted_price / total_volume

def update_price(price, source="WEBSOCKET"):
    global current_price, last_update, last_source
    price = float(price)
    if not math.isfinite(price) or price <= 0:
        return
    now = time.time()
    bucket = int(now // 60) * 60
    with lock:
        current_price = price
        last_update = now
        last_source = source
        ticks.append({"time": now, "price": price})
```

```
        if candles and int(candles[-1]["time"]) == bucket:
            c = candles[-1]
            c["high"] = max(c["high"], price)
            c["low"] = min(c["low"], price)
            c["close"] = price
        else:
            candles.append({
                "time": bucket, "open": price, "high": price,
                "low": price, "close": price, "volume": 0.0
            })

def record_trade(side, size):
    now = time.time()
    side = str(side or "").upper()
    try:
        size = float(size)
    except (TypeError, ValueError):
        return
    if size <= 0 or side not in ("BUY", "SELL"):
        return
    with lock:
        trade_flow.append({"time": now, "side": side, "size": size})
        if candles:
            bucket = int(now // 60) * 60
            if int(candles[-1]["time"]) == bucket:
                candles[-1]["volume"] += size

def load_history():
    global last_error
    try:
        response = session.get(
            f"{REST_URL}/products/{PRODUCT_ID}/candles",
            params={"granularity": 60}, timeout=15
        )
        response.raise_for_status()
        history = []
        for item in response.json():
            if len(item) < 6:
                continue
            history.append({
                "time": int(item[0]), "low": float(item[1]),
                "high": float(item[2]), "open": float(item[3]),
                "close": float(item[4]), "volume": float(item[5])
```

```
            })
        history.sort(key=lambda x: x["time"])
        with lock:
            existing = {int(c["time"]): dict(c) for c in history}
            for c in candles:
                existing[int(c["time"])] = dict(c)
            candles.clear()
            candles.extend(sorted(existing.values(), key=lambda x: x["time"])[-MAX_CANDLES:])
            last_error = ""
    except Exception as exc:
        with lock:
            last_error = "Historial: " + str(exc)

def force_coinbase_update():
    global last_error
    try:
        response = session.get(f"{REST_URL}/products/{PRODUCT_ID}/ticker", timeout=10)
        response.raise_for_status()
        update_price(float(response.json()["price"]), "REST MANUAL")
        with lock:
            last_error = ""
        return True
    except Exception as exc:
        with lock:
            last_error = str(exc)
        return False

def on_open(ws):
    global ws_connected, last_error
    try:
        ws.send(json.dumps({"type": "subscribe", "product_ids": [PRODUCT_ID], "channel":
"ticker"}))
        ws.send(json.dumps({"type": "subscribe", "product_ids": [PRODUCT_ID], "channel":
"market_trades"}))
        with lock:
            ws_connected = True
            last_error = ""
    except Exception as exc:
        with lock:
            ws_connected = False
            last_error = str(exc)

def on_message(ws, message):
```

```
    global last_error
    try:
        data = json.loads(message)
        if data.get("channel") == "ticker":
            for event in data.get("events", []):
                for ticker in event.get("tickers", []):
                    if ticker.get("product_id") == PRODUCT_ID:
                        update_price(ticker["price"], "WEBSOCKET")
        elif data.get("channel") == "market_trades":
            for event in data.get("events", []):
                for trade in event.get("trades", []):
                    if trade.get("product_id") == PRODUCT_ID:
                        record_trade(trade.get("side"), trade.get("size"))
        with lock:
            last_error = ""
    except Exception as exc:
        with lock:
            last_error = str(exc)

def on_error(ws, error):
    global ws_connected, last_error
    with lock:
        ws_connected = False
        last_error = str(error)

def on_close(ws, code, message):
    global ws_connected
    with lock:
        ws_connected = False

def websocket_worker():
    global ws_connected, last_error
    while True:
        try:
            ws = websocket.WebSocketApp(WS_URL, on_open=on_open,
on_message=on_message, on_error=on_error, on_close=on_close)
            ws.run_forever(ping_interval=20, ping_timeout=10)
        except Exception as exc:
            with lock:
                ws_connected = False
                last_error = str(exc)
        time.sleep(5)
```

```
def rest_fallback_worker():
    global last_error
    while True:
        try:
            with lock:
                updated = last_update
            if time.time() - updated > 8:
                response = session.get(f"{REST_URL}/products/{PRODUCT_ID}/ticker", timeout=10)
                response.raise_for_status()
                update_price(float(response.json()["price"]), "REST RESPALDO")
        except Exception as exc:
            with lock:
                last_error = str(exc)
        time.sleep(5)

def start_background_services():
    global services_started
    with lock:
        if services_started:
            return
        services_started = True
    threading.Thread(target=load_history, daemon=True).start()
    threading.Thread(target=websocket_worker, daemon=True).start()
    threading.Thread(target=rest_fallback_worker, daemon=True).start()

def momentum_pct(price, history, seconds, now):
    target = now - seconds
    old = next((t for t in reversed(history) if float(t["time"]) <= target), None)
    if old is None or target - float(old["time"]) > 4:
        return None
    old_price = float(old["price"])
    if old_price <= 0:
        return None
    return (price / old_price - 1) * 100

def calculate_indicators(price, history, tick_data, flow_data):
    closes = [float(c["close"]) for c in history if c.get("close") is not None]
    ema9 = ema(closes, 9) if len(closes) >= 9 else None
    ema21 = ema(closes, 21) if len(closes) >= 21 else None
    rsi_value = rsi(closes, 14)
    vwap_value = vwap_from_candles(history)

    now = time.time()
```

```
m5 = momentum_pct(price, tick_data, 5, now)
m15 = momentum_pct(price, tick_data, 15, now)
m30 = momentum_pct(price, tick_data, 30, now)

recent = [t for t in flow_data if float(t["time"]) >= now - 60]
buy = sum(float(t["size"]) for t in recent if t["side"] == "BUY")
sell = sum(float(t["size"]) for t in recent if t["side"] == "SELL")
total = buy + sell
buy_pct = buy / total * 100 if total > 0 else None
sell_pct = sell / total * 100 if total > 0 else None

score = 0.0
max_score = 0.0

if ema9 is not None and ema21 is not None:
    max_score += 3.0
    if ema9 > ema21: score += 3.0
    elif ema9 < ema21: score -= 3.0

if vwap_value is not None:
    max_score += 2.0
    if price > vwap_value: score += 2.0
    elif price < vwap_value: score -= 2.0

if rsi_value is not None:
    max_score += 1.5
    if rsi_value >= 52: score += 1.5
    elif rsi_value <= 48: score -= 1.5

for value, weight in ((m5, 0.5), (m15, 1.0), (m30, 1.0)):
    if value is not None:
        max_score += weight
        if value > 0: score += weight
        elif value < 0: score -= weight

if buy_pct is not None:
    max_score += 1.5
    if buy_pct >= 52: score += 1.5
    elif buy_pct <= 48: score -= 1.5

normalized_score = score / max_score if max_score > 0 else 0.0

if normalized_score >= 0.15:
```

```
        direction, signal = "ALCISTA", "SUBE"
    elif normalized_score <= -0.15:
        direction, signal = "BAJISTA", "BAJA"
    else:
        direction, signal = "INDECISO", "INDECISO"

    bullish = max(5, min(95, round(50 + normalized_score * 40)))
    bearish = 100 - bullish
    confidence = max(10, min(95, round(abs(normalized_score) * 100)))

    usable = [value for value in (m5, m15, m30) if value is not None]
    average = sum(usable) / len(usable) if usable else 0.0
    projected_change = max(-0.0025, min(0.0025, average / 100 * 2))
    prediction = price * (1 + projected_change)

    return {
        "direction": direction, "signal": signal,
        "score": round(score, 2), "normalized_score": round(normalized_score, 4),
        "ema9": round(ema9, 2) if ema9 is not None else None,
        "ema21": round(ema21, 2) if ema21 is not None else None,
        "rsi": round(rsi_value, 2) if rsi_value is not None else None,
        "vwap": round(vwap_value, 2) if vwap_value is not None else None,
        "buy_pct": round(buy_pct, 1) if buy_pct is not None else None,
        "sell_pct": round(sell_pct, 1) if sell_pct is not None else None,
        "buy_volume": round(buy, 6), "sell_volume": round(sell, 6),
        "momentum_5s_pct": round(m5, 5) if m5 is not None else None,
        "momentum_15s_pct": round(m15, 5) if m15 is not None else None,
        "momentum_30s_pct": round(m30, 5) if m30 is not None else None,
        "prediction": round(prediction, 2),
        "bullish_probability": bullish, "bearish_probability": bearish,
        "confidence": confidence
    }

def get_api_data():
    with lock:
        price = current_price
        history = [dict(c) for c in candles]
        tick_data = [dict(t) for t in ticks]
        flow_data = [dict(t) for t in trade_flow]
        updated = last_update
        error = last_error
        source = last_source
        socket_status = ws_connected
```

```
    now = time.time()
    live = bool(price is not None and updated > 0 and now - updated <= STALE_SECONDS)
    countdown = 900 - (int(now) % 900)

    if price is None:
        return {
            "ok": False, "connected": False, "status": "ESPERANDO COINBASE",
            "price": None, "candles": history, "countdown": countdown, "error": error
        }

    indicators = calculate_indicators(price, history, tick_data, flow_data)
    return {
        "ok": True, "connected": live, "websocket_connected": socket_status,
        "status": "EN VIVO" if live else "SIN DATOS RECIENTES",
        "price": round(price, 2), "candles": history[-MAX_CANDLES:],
        "countdown": countdown, "last_update": updated, "source": source,
        "error": error, **indicators
    }

@app.route("/")
def home():
    start_background_services()
    return render_template_string(HTML)

@app.route("/api")
def api():
    start_background_services()
    return jsonify(get_api_data())

@app.route("/refresh")
def refresh():
    start_background_services()
    success = force_coinbase_update()
    data = get_api_data()
    data["manual_refresh"] = success
    return jsonify(data)

@app.route("/health")
def health():
    with lock:
        price = current_price
        updated = last_update
```

```
        error = last_error
        source = last_source
        count = len(candles)
        socket_status = ws_connected
    live = bool(price is not None and updated > 0 and time.time() - updated <=
STALE_SECONDS)
    return jsonify({
        "ok": True, "coinbase_live": live, "websocket_connected": socket_status,
        "price": price, "last_update": updated, "source": source, "candles": count, "error": error
    })

HTML = r"""
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Bitcoin Radar Live</title>
<style>
*{box-sizing:border-box}
body{margin:0;padding:14px;background:#080d18;color:#edf2ff;font-family:Arial,sans-serif}
main{max-width:1000px;margin:auto}
.header{display:flex;justify-content:space-between;align-items:center;margin-bottom:14px}
h1{font-size:20px;margin:0}
.status{font-size:11px;color:#22d69a;font-weight:bold}
.price-grid{display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:9px}
.price-card,.card,.probability{background:#111a2b;border:1px solid
#263650;border-radius:13px;padding:14px;margin-bottom:10px;min-width:0}
.label{font-size:11px;color:#91a2c1;letter-spacing:.6px}
.price{font-size:clamp(22px,5vw,34px);font-weight:bold;margin:9px 0;overflow-wrap:anywhere}
.value{font-size:19px;font-weight:bold;margin-top:6px}
.up{color:#22d69a}
.down{color:#ff6475}
.neutral{color:#f4c76b}
.small{font-size:11px;color:#91a2c1;line-height:1.6}
.row{display:flex;justify-content:space-between;align-items:center;gap:10px}
.probability{margin-top:0}
.probbar,.bar{display:flex;height:12px;border-radius:8px;overflow:hidden;background:#26334e;
margin:12px 0}
.prob-up,.buybar{background:#22d69a}
.prob-down,.sellbar{background:#ff6475}
.signal{font-size:25px;font-weight:800;margin:8px 0}
button{border:0;border-radius:8px;padding:10px
```

```
13px;color:white;background:#294a83;font-weight:bold}
.controls{display:flex;gap:8px;flex-wrap:wrap;margin:12px 0}
.controls button{min-width:48px}
canvas{display:block;width:100%;height:310px;touch-action:none}
.tag{display:inline-block;padding:4px
7px;background:#202d45;border-radius:6px;font-size:10px;color:#bdcbea}
@media(max-width:600px){body{padding:9px}.price-card{padding:10px}.price{font-size:22px}ca
nvas{height:280px}}
</style>
</head>
<body>
<main>
<div class="header"><h1>BITCOIN RADAR LIVE</h1><div id="status"
class="status">CONECTANDO...</div></div>
<div class="price-grid">
  <section class="price-card"><div class="label">PRECIO ACTUAL · COINBASE</div><div
id="price" class="price">--</div><div class="small">BTC / USD · <span id="source"
class="tag">ESPERANDO</span></div></section>
  <section class="price-card"><div class="label">PRECIO PREDICHO · 15
MINUTOS</div><div id="prediction" class="price">--</div><div id="predDirection"
class="small">ESPERANDO</div></section>
</div>
<section class="probability">
  <div class="label">PORCENTAJE DE DIRECCIÓN</div>
  <div class="row" style="margin-top:12px">
    <div><div class="label">ALCISTA</div><div id="bullPct" class="value up">--</div></div>
    <div style="text-align:right"><div class="label">BAJISTA</div><div id="bearPct"
class="value down">--</div></div>
  </div>
  <div class="probbar"><div id="bullBar" class="prob-up" style="width:50%"></div><div
id="bearBar" class="prob-down" style="width:50%"></div></div>
  <div class="row"><div id="direction" class="signal neutral">INDECISO</div><div
id="confidence" class="small">FUERZA --</div></div>
  <div class="small">Porcentajes estimados mediante indicadores técnicos. No representan
probabilidades garantizadas.</div>
</section>
<section class="card">
  <div class="label">COMPRADORES Y VENDEDORES · ÚLTIMOS 60 SEGUNDOS</div>
  <div class="bar"><div id="buyBar" class="buybar" style="width:50%"></div><div id="sellBar"
class="sellbar" style="width:50%"></div></div>
  <div class="row">
    <div><div class="label">COMPRADORES</div><div id="buyPct" class="value
up">--</div></div>
```

```
      <div style="text-align:right"><div class="label">VENDEDORES</div><div id="sellPct"
class="value down">--</div></div>
   </div>
   <div id="flowNote" class="small">Esperando operaciones.</div>
</section>
<section class="card">
   <div class="row">
      <div><div class="label">VELAS DE 1 MINUTO</div><div class="small"><span
style="color:#69a8ff">EMA 9</span> · <span style="color:#f4a43b">EMA
21</span></div></div>
      <button id="refreshButton" onclick="manualRefresh()">ACTUALIZAR</button>
   </div>
   <div class="controls">
      <button onclick="zoomIn()">+</button><button onclick="zoomOut()">−</button><button
onclick="resetZoom()">AJUSTAR</button><button onclick="goLive()">EN VIVO</button>
   </div>
   <canvas id="chart" width="1000" height="430"></canvas>
   <div class="small">Arrastra con un dedo para mover la gráfica. Usa dos dedos para acercar
o alejar.</div>
</section>
<section class="card">
   <div class="row">
      <div><div class="label">CIERRE DEL BLOQUE ACTUAL</div><div id="countdown"
class="signal">15:00</div></div>
      <div><div class="label">WEBSOCKET</div><div id="socket" class="value">--</div></div>
   </div>
   <div id="updated" class="small"></div>
   <div id="error" class="small"></div>
</section>
</main>
<script>
const $ = id => document.getElementById(id);
const money = value => value == null ? "--" :
Number(value).toLocaleString("en-US",{style:"currency",currency:"USD",maximumFractionDigit
s:2});
const pct = value => value == null ? "--" : Number(value).toFixed(1)+"%";
function formatTime(seconds){ const m=Math.floor(seconds/60); const s=seconds%60; return
String(m).padStart(2,"0")+":"+String(s).padStart(2,"0"); }
let chartData=[]; let zoomLevel=1; let chartOffset=0;
function calcEmaSeries(values,period){
   if(!values.length)return [];
   const alpha=2/(period+1); let result=values[0]; const output=[result];
   for(let i=1;i<values.length;i++){ result=alpha*values[i]+(1-alpha)*result; output.push(result); }
```

```
    return output;
}
function zoomIn(){ zoomLevel=Math.min(8,zoomLevel*1.3); drawChart(chartData); }
function zoomOut(){ zoomLevel=Math.max(.5,zoomLevel/1.3); drawChart(chartData); }
function resetZoom(){ zoomLevel=1; chartOffset=0; drawChart(chartData); }
function goLive(){ chartOffset=0; drawChart(chartData); }
function drawChart(data){
    chartData=data||[]; const canvas=$("chart"); const ctx=canvas.getContext("2d");
    const W=canvas.width; const H=canvas.height; ctx.clearRect(0,0,W,H);
    if(!chartData.length){ ctx.fillStyle="#91a2c1"; ctx.font="16px Arial"; ctx.fillText("ESPERANDO
VELAS...",30,40); return; }
    const count=Math.max(12,Math.min(100,Math.floor(100/zoomLevel)));
    const centerSlot=Math.floor(count/2); const
visibleCount=Math.min(chartData.length,centerSlot+1);
    const maxOffset=Math.max(0,chartData.length-1);
chartOffset=Math.max(0,Math.min(chartOffset,maxOffset));
    const end=chartData.length-chartOffset; const start=Math.max(0,end-visibleCount);
    const visible=chartData.slice(start,end); const closes=chartData.map(c=>Number(c.close));
    const ema9=calcEmaSeries(closes,9).slice(start,end); const
ema21=calcEmaSeries(closes,21).slice(start,end);
    const values=visible.flatMap(c=>[Number(c.high),Number(c.low)]).concat(ema9,ema21);
    let min=Math.min(...values); let max=Math.max(...values);
    const padding=Math.max((max-min)*.12,2); min-=padding; max+=padding;
    const left=85; const right=W-12; const top=25; const bottom=H-30; const usable=bottom-top;
    const y=price => bottom-((price-min)/(max-min))*usable;
    ctx.strokeStyle="#263650"; ctx.lineWidth=1; ctx.font="13px Arial";
    for(let i=0;i<5;i++){
        const yy=top+usable*i/4; ctx.beginPath(); ctx.moveTo(left,yy); ctx.lineTo(right,yy);
ctx.stroke();
        const value=max-(max-min)*i/4; ctx.fillStyle="#91a2c1"; ctx.fillText(money(value),3,yy-3);
    }
    const step=(right-left)/count; const cw=Math.max(3,Math.min(18,step*.62));
    function candleX(index){ return left+(centerSlot-(visible.length-1-index)+.5)*step; }
    visible.forEach((c,i)=>{
        const x=candleX(i); const open=Number(c.open); const close=Number(c.close); const
high=Number(c.high); const low=Number(c.low);
        const up=close>=open; const color=up?"#22d69a":"#ff6475";
        ctx.strokeStyle=color; ctx.fillStyle=color; ctx.lineWidth=2;
        ctx.beginPath(); ctx.moveTo(x,y(high)); ctx.lineTo(x,y(low)); ctx.stroke();
        const oy=y(open); const cy=y(close);
        ctx.fillRect(x-cw/2,Math.min(oy,cy),cw,Math.max(2,Math.abs(cy-oy)));
    });
    function drawLine(series,color){
```

```
        if(!series.length)return; ctx.beginPath(); ctx.strokeStyle=color; ctx.lineWidth=2;
        series.forEach((value,i)=>{ const x=candleX(i); if(i===0)ctx.moveTo(x,y(value)); else
ctx.lineTo(x,y(value)); });
        ctx.stroke();
    }
    drawLine(ema9,"#69a8ff"); drawLine(ema21,"#f4a43b");
    const last=chartData[chartData.length-1];
    if(last && chartOffset===0){
        const price=Number(last.close); const yy=y(price);
        ctx.setLineDash([5,5]); ctx.strokeStyle="#d6e2ff"; ctx.beginPath(); ctx.moveTo(left,yy);
ctx.lineTo(right,yy); ctx.stroke(); ctx.setLineDash([]);
    }
}
let pointers=new Map(); let initialDistance=0; let initialZoom=1; let previousX=0;
function distance(a,b){ return Math.hypot(a.x-b.x,a.y-b.y); }
const canvas=$("chart");
canvas.addEventListener("pointerdown",e=>{
    canvas.setPointerCapture(e.pointerId); pointers.set(e.pointerId,{x:e.clientX,y:e.clientY});
    if(pointers.size===1) previousX=e.clientX;
    if(pointers.size===2){ const p=[...pointers.values()]; initialDistance=distance(p[0],p[1]);
initialZoom=zoomLevel; }
});
canvas.addEventListener("pointermove",e=>{
    if(!pointers.has(e.pointerId))return;
    pointers.set(e.pointerId,{x:e.clientX,y:e.clientY});
    if(pointers.size===2){
        const p=[...pointers.values()]; const d=distance(p[0],p[1]);
        if(initialDistance>0){ zoomLevel=Math.max(.5,Math.min(8,initialZoom*d/initialDistance));
drawChart(chartData); }
    }else if(pointers.size===1){
        const delta=e.clientX-previousX; previousX=e.clientX;
        const count=Math.max(12,Math.min(100,Math.floor(100/zoomLevel)));
        const step=canvas.clientWidth/count;
        chartOffset+=Math.round(-delta/Math.max(1,step));
        chartOffset=Math.max(0,Math.min(chartOffset,Math.max(0,chartData.length-1)));
        drawChart(chartData);
    }
});
function releasePointer(e){ pointers.delete(e.pointerId); if(pointers.size<2) initialDistance=0; }
canvas.addEventListener("pointerup",releasePointer);
canvas.addEventListener("pointercancel",releasePointer);

let requestInProgress=false;
```

```
async function refreshData(manual=false){
  if(requestInProgress)return; requestInProgress=true;
  const button=$("refreshButton");
  if(manual){ button.disabled=true; button.textContent="ACTUALIZANDO"; }
  try{
    const response=await
fetch((manual?"/refresh":"/api")+"?_="+Date.now(),{cache:"no-store"});
    if(!response.ok) throw new Error("HTTP "+response.status);
    const d=await response.json();
    if(d.price!==null && d.price!==undefined){
      $("price").textContent=money(d.price);
      $("prediction").textContent=money(d.prediction);
      $("predDirection").textContent=d.signal;
      $("bullPct").textContent=pct(d.bullish_probability);
      $("bearPct").textContent=pct(d.bearish_probability);
      $("bullBar").style.width=d.bullish_probability+"%";
      $("bearBar").style.width=d.bearish_probability+"%";
      const
directionClass=d.direction==="ALCISTA"?"up":d.direction==="BAJISTA"?"down":"neutral";
      $("direction").className="signal "+directionClass;
      $("direction").textContent=d.direction;
      $("confidence").textContent="FUERZA "+d.confidence+"%";
      $("predDirection").className=d.signal==="SUBE"?"small up":d.signal==="BAJA"?"small
down":"small neutral";
      $("buyPct").textContent=pct(d.buy_pct);
      $("sellPct").textContent=pct(d.sell_pct);
      if(d.buy_pct!==null && d.buy_pct!==undefined){
        $("buyBar").style.width=d.buy_pct+"%";
        $("sellBar").style.width=d.sell_pct+"%";
        $("flowNote").textContent="Volumen comprador: "+d.buy_volume+" BTC · vendedor:
"+d.sell_volume+" BTC";
      }else{
        $("flowNote").textContent="Esperando suficiente flujo de operaciones.";
      }
      drawChart(d.candles);
    }
    $("countdown").textContent=formatTime(d.countdown||0);
    $("status").textContent=d.connected?"● EN VIVO":"● SIN DATOS";
    $("status").style.color=d.connected?"#22d69a":"#ff6475";
    $("source").textContent=d.source||"NINGUNA";
    $("source").textContent=d.source||"NINGUNA";
    $("socket").textContent=d.websocket_connected?"CONECTADO":"DESCONECTADO";
    if(d.last_update){
```

```
            $("updated").textContent="Último dato: "+new
Date(d.last_update*1000).toLocaleTimeString();
        }
        $("error").textContent=d.error||"";
    }catch(err){
        $("status").textContent="ERROR DE CONEXIÓN";
        $("error").textContent=String(err);
    }finally{
        requestInProgress=false;
        if(manual){ button.disabled=false; button.textContent="ACTUALIZAR"; }
    }
}
function manualRefresh(){ refreshData(true); }
refreshData();
setInterval(()=>{ refreshData(false); },1000);
</script>
</body>
</html>
"""

if __name__ == "__main__":
    start_background_services()
    app.run(host="0.0.0.0", port=10000, threaded=True)
```